Part - 01

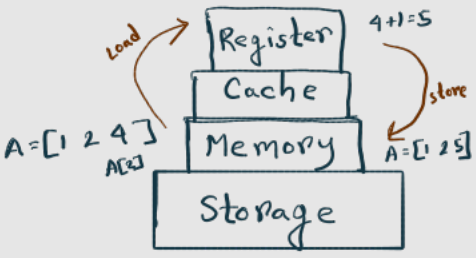


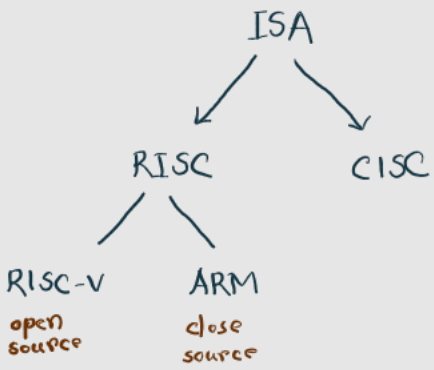
int a = b + c
x5 x6 x7
a = b - c

add x5, x6, x7
x5 = x6 + x7
a b c

sub x5, x6, x7
a = b - c

10101.....





ISA (Instruction Set Architecture) - Hardware Software Interface

RISC-V : A type of ISA. Has modular design

Reduced Instruction set computer

Base ISA + Optional Extention
[Basic instructions] [Additional instructions can be included]

Unit	Bits
Byte	8
Half word	16
Word	32
Double word	64

REGISTERS

Register = a small, fast storage space the CPU uses for data it needs right now or uses often. RISC-V has **32 registers**, each **64 bits**. (64-bit architecture = the CPU handles 64 bits at a time.)

> **Design principle 2: Smaller is faster.** 32 × 64 is the best size to get the clock period we want (factors like the register file).
> **Design principle 1: Simplicity favors regularity.** All arithmetic operations have the same form.

x0
x1
x2
.
.
.
x31 = _____

Register	Functionality
x0	Constant value 0
x1	Return address
x2	Stack pointer (address of the top of the stack)
x3	Global pointer
x4	Thread pointer
x5 - x7	**Temporaries**
x8	Frame pointer
x9	**Saved register**
x10 - x11	Function argument / return value
x12 - x17	Function arguments
x18 - x27	**Saved registers**
x28 - x31	**Temporaries**

RISC-V follows this convention to increase readability. Orange = temporaries. Green = saved registers.

x0 = 0

Component	Example	Size
Register name	x20, x21	5 bit
Register size / data value	value inside x20	64 bit
Instruction	add x20, x20, x21	32 bit

* Memory location এর জন্য divide by 8

Operand : Part of instruction to manupilate on

1. Register operand (small capacity + fast)
2. Constant / immediate
3. Memory operand (Large capacity + slow)

① Register operand : Does arithmetic instruction.

Ques: $f = (g+h) - (i+j)$ to assembly code

```
add  x5, x20, x21    // x5 = g+h
add  x6, x22, x23    // x6 = i+j
sub  x19, x5, x6
```

f = x19
g = x20
h = x21
i = x22
j = x23

② Immediate / Constant Operand : Add constant data

- No subi available (Design principle 2)
- avoids a load instruction

```
g = f+5
addi  x22, x21, 5
```
↑ decimal

```
g = 1+2
addi  x22, 1, 2   ✗
```

```
g = f-5
addi x22, x21, -5
```

* Why `addi x5, x5, 0` is faster than `add x5, x5, x0` ?

`x5 = x5 + 0` ↑ hardwire

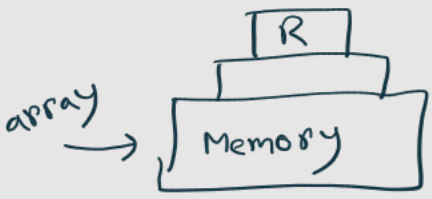
`x5 = x5 + x0`

③ Memory Operand :

MEMORY STORAGE

1. Arrays (composite data) live in **main memory**. To do arithmetic, **load** data from memory to a register. Then **store** the result back.
2. Memory is **byte addressed**: each slot has 8 bits of data.
3. **No alignment instruction.** A word starts at a multiple of 4. A double word starts at a multiple of 8.
4. **Endianness** = the order in which memory stores the bytes of data.

Type	Rule
Big endian	MSB goes in the lowest location
Little endian (RISC-V)	LSB goes in the lowest location. Better for addition, because we start adding from the LSB.



Question : Store $(10)_{10}$ in a RISC-V machine / memory

Little endian :

0000	0000	1010
0001	0000	0000
0002	0000	0000
0003	0000	0000
0004	0000	0000
0005	0000	0000
0006	0000	0000
0007	0000	0000

$(10)_{10} \Rightarrow$

MSB ← 0000 0000 0000 0000
0000 0000 0000 0000
0000 0000 0000 0000
0000 0000 0000 1010 → LSB

*Benefit : Easier to add with LSB

Big endian :

0000	0000	0000
0001	0000	0000
0002	0000	0000
0003	0000	0000
0004	0000	0000
0005	0000	0000
0006	0000	0000
0007	0000	1010

$(10)_{10} \Rightarrow$

MSB ← 0000 0000 0000 0000
0000 0000 0000 0000
0000 0000 0000 0000
0000 0000 0000 1010 → LSB

A = [1 2 3]
A[0] A[1] A[2]
000 008 016

A[100]

000	A[0] ← x22
008	A[1] } 16
016	A[2]

16[x22]
offset

→ x22+16
A[0]+16
A[0]+8×n
doubleword
A[0]+4×n
word

Question : Store 0xabcdef12 in RISC-V machine starting

(abcdef12) = 1010 1011 1100 1101 1110 1111 0001 0010
a b c d e f 1 2

0001	0010
1110	1111
1100	1101
1010	1011

Load / store syntax : Array

ld ,
rd rs

ld x5, x6

sd ,
rs1 rs2

sd x5, x6

Example :

ld x21, 64[x20]
rd offset rs

lw x21, 64[x20]

sd x21, 40[x20]
rs1 rs2

A = [10, 20, 30, 40]

A[0] A[1] A[2] A[3]

Memory address : 1000 1008 1016 1024

Question: Convert int n = 0 to assembly code. Supose n is in x21

```
addi x21, x0, 0      // n = x0+0
```

n = x21

[int n = 3] `addi x21, x0, 3` // n = x0 + 3

Question: Convert g = h + A[8] to assembly code. Suppose g is in x20, h in x21 and base of A in x22

```
ld  x5, 64[x22]      // A[8]
add x20, x21, x5     // g = h + A[8]
```

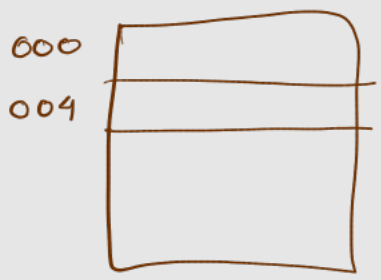
(8×n)

g = x20
h = x21
Base_A = x22 ↑ A[0]

Let's assume that A is an array of 100 doublewords and that the compiler has associated the variables g and h with the registers x20 and x21 as before. Let's also assume that the starting address, or *base address*, of the array is in x22. Compile this C assignment statement:

```
g = h + A[8];
```

Same



Same question but with '100 word'

4×n

```
lw  x5, 32[x22]      // A[8]
add x20, x21, x5     // g=h+A[8]
```

Question: Convert A[12]=h+A[6] to assembly code. h is in x21 and base of A is in x22

```
ld  x5, 48[x22]      // A[8]
add x5, x22, x5      // x5= x22+x5
sd  x5, 96[x22]
```

8×n
= 8×6
= 48

h = x21
Base_A = x22
A[0]

Question: Convert A[12]=1+A[6] to assembly code. h is in x21 and base of A is in x22

```
ld  x5, 48[x22]
addi x5, x5, 1       // x5 = A[6]+1
sd  x5, 96[x22]
```

Question: Convert A[12]=1+A[g] to assembly code. h is in x21 and base of A is in x22

Main soln next video

x22 + g×8

Sign Extension:

Size	Load (memory to register)	Store (register to memory)
Double word (64 bit)	ld	sd
Word (32 bit)	lw	sw
Half word (16 bit)	lh	sh
Byte (8 bit)	lb	sb

$(2)_{10} = (10)$

LBU x22, 10[x21]

unsigned

ldu x22, 10[x21]

$(+2)_{10} = (010)_2$

$= 00000010$

$(-2)_{10} =$

$(+2)_{10} = 00000010$

$(-2)_{10} = 11111110$

Unsigned : $0 - 2^n - 1$ [0 to 255]

Signed : -2^{n-1} to $2^{n-1} - 1$ [-128 to 127]